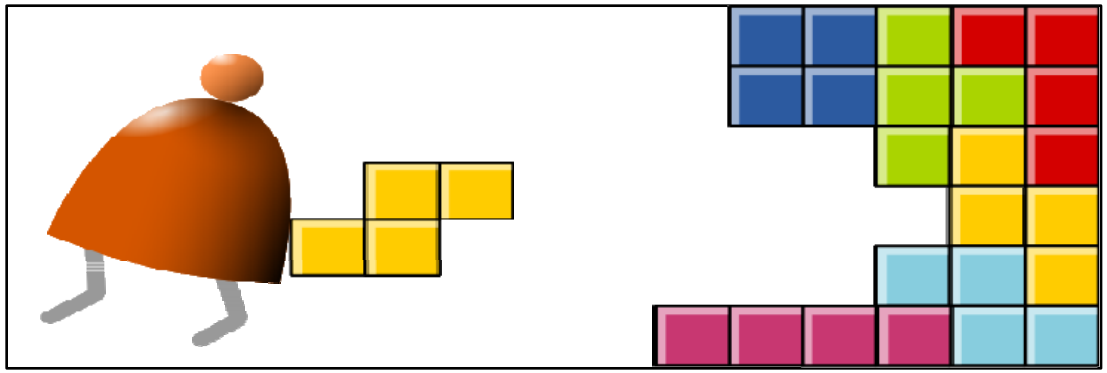
Boolean Arithmetic



Building a Modern Computer From First Principles

www.nand2tetris.org

WOOSONG UNIVERSITY
Kazakhstan

Boolean Arithmetic

- Building a computer requires mastering binary math.
- We will move from binary basics to designing an Arithmetic Logic Unit (ALU).
- Focus: addition, overflow, negative numbers, adder circuits.



WOOSONG UNIVERSITY
Kazakhstan

Why Boolean Arithmetic?

- All computer operations reduce to binary math.
- Hardware circuits must perform arithmetic efficiently.
- Forms the basis for the ALU



WOOSONG UNIVERSITY
Kazakhstan

Decimal vs. Binary Systems

Decimal	4-Bit Binary
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111

- Decimal: Base-10 (digits 0–9)
- Binary: Base-2 (digits 0 and 1)
- Example: 5 (decimal) = 101 (binary)
- Hardware registers store only binary values.



WOOSONG UNIVERSITY
Kazakhstan

How Many Bits Do You Need?

- The number of bits you need depends on the largest number you want to represent. Each bit you add doubles the number of possible values.
- **Simple Rule:** The number of bits corresponds to the power of 2 that is greater than or equal to your number.

Number of Bits	Total Possibilities	Decimal Range	Example
1	$2^1 = 2$	0 - 1	0, 1
2	$2^2 = 4$	0 - 3	00 to 11
3	$2^3 = 8$	0 - 7	000 to 111
4	$2^4 = 16$	0 - 15	0000 to 1111

- **Decimal 7:** The smallest power of 2 that is ≥7 is $2^3 = 8$. Therefore, you need **3 bits** to represent **7**.
- **Decimal 14:** The smallest power of 2 that is ≥14 is $2^4 = 16$. Therefore, you need **4 bits** to represent **14**.



WOOSONG UNIVERSITY
Kazakhstan

Counting systems

quantity	decimal	binary	3-bit register
	0	0	000
*	1	1	001
**	2	10	010
***	3	11	011
****	4	100	100
*****	5	101	101
******	6	110	110
*******	7	111	111
********	8	1000	overflow
*********	9	1001	overflow
**********	10	1010	overflow



WOOSONG UNIVERSITY
Kazakhstan

Rationale and Conversion

Binary to Decimal

For a binary number with digits
$b_{n_1}\, b_{n_2} \ldots b_1\, b_0$

$$\text{Decimal} = \sum_{i=0}^{n-1} b_i \cdot 2^i$$

$(10011)_2 = (1\times2^4)+(0\times2^3)+(0\times2^2)+(1\times2^1)+(1\times2^0)$

$= (1\times16)+(0\times8)+(0\times4)+(1\times2)+(1\times1)$

$= 16+0+0+2+1$

$= 19$

Decimal number = $(b_{n-1} \times 2^{n-1}) + (b_{n-2} \times 2^{n-2}) + \ldots + (b_1 \times 2^1) + (b_0 \times 2^0)$

- $b_0 \rightarrow$ **rightmost digit** (least significant bit, LSB).
- $b_{n_1} \rightarrow$ **leftmost digit** (most significant bit, MSB)



Elements of Computing Systems, Nisan & Schocken, MIT Press, www.nand2tetris.org , Chapter 2: *Boolean Arithmetic*
https://sathee.iitk.ac.in/article/maths/maths-binary-to-decimal-conversion/

Rationale and Conversion

Decimal to Binary

To convert the decimal number **19** into binary, you can use the same method of successive division by 2.

Dividend	Divisor	Quotient	Remainder (Binary Digit)
19	2	9	**1**
9	2	4	**1**
4	2	2	**0**
2	2	1	**0**
1	2	0	**1**

2	**19**	**1**
2	**9**	**1**
2	**4**	**0**
2	**2**	**0**
	1	

Now, read the remainders from the bottom up to get the binary number.

The binary representation of 19 is **10011**.



WOOSONG UNIVERSITY
Kazakhstan

Binary Addition and Overflow

- Addition rules:

  0+0=0,

  0+1=1,

  1+0=1,

  1+1=0 (carry 1)

- Overflow occurs when carry exceeds available bits.

quantity	decimal	binary	3-bit register
	0	0	000
*	1	1	001
**	2	10	010
***	3	11	011
****	4	100	100
*****	5	101	101
******	6	110	110
*******	7	111	111
********	8	1000	overflow
*********	9	1001	overflow
**********	10	1010	overflow



Binary addition

- Assuming a 4-bit system:

```
  0 0 0 1          1 1 1 1
    1 0 0 1          1 0 1 1
  + 0 1 0 1        + 0 1 1 1
  ---------        ---------
  0 1 1 1 0        1 0 0 1 0
```

no overflow **overflow**

- Algorithm: exactly the same as in decimal addition
- Overflow (Most Significant Bit (MSB carry)) has to be dealt with.



WOOSONG UNIVERSITY
Kazakhstan

Two's Complement: The Language of Signed Numbers

- The most common method to represent signed integers in computer systems.
- Works with Boolean arithmetic (binary digits 0s and 1s).
- Represents both positive and negative numbers.
- Ensures a single representation of zero.
- It allows addition and subtraction to be performed using the same fundamental hardware circuit, which simplifies computer design.



WOOSONG UNIVERSITY
Kazakhstan

Representing negative numbers (4-bit system)

- In a 4-bit system, we use the first bit to determine if a number is positive or negative. This is called Two's Complement.
  - Positive numbers start with **0**.
  - Negative numbers start with **1**.

How to Convert a Positive Number to a Negative Number

- To get the negative form of a number, we follow a simple two-step rule. Let's use 5 (binary 0101) as an example to get -5.
  - Flip the bits: Change all the 0s to 1s and all the 1s to 0s. 0101 becomes 1010. (Take the 1's complement (invert all bits).
  - Add one: Add 1 to the result from the previous
    - 1010 + 0001 = 1011
  - So, -5 is represented as 1011.

0	0000		
1	0001	1111	-1
2	0010	1110	-2
3	0011	1101	-3
4	0100	1100	-4
5	0101	1011	-5
6	0110	1010	-6
7	0111	1001	-7
		1000	-8

Represent negative number -x using the positive number: $2^n - x$

4-Bit Binary	Decimal (Signed / Two's Complement)	
0000	0	
0001	1	
0010	2	
0011	3	
0100	4	
0101	5	
0110	6	
0111	7	
1000	-8	8
1001	-7	9
1010	-6	10
1011	-5	11
1100	-4	12
1101	-3	13
1110	-2	14
1111	-1	15

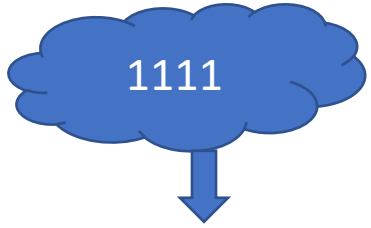
WOOSONG UNIVERSITY
Kazakhstan

Why It's Useful (Hardware & Arithmetic)

- **Uniform Arithmetic:** Addition and subtraction use the same hardware circuit.
- **Simplifies Hardware Design:** No need for separate subtraction logic.
- **Overflow Handling:** Extra carry bit is discarded automatically.
- **Efficient & Widely Used:** Almost all modern CPUs use it.



Computing -x



Input: x

Output: -x(in 2s complement)

If we solve this then we know how to subtract!

- *y-x =y +(-x)* *subtraction is just addition of a negative number.*

- **Idea:** $2^n - x = (2^n - 1) - x + 1$
- **Example:** input -5

$$\begin{array}{rr} & 1111 \\ - & 0101 \\ \hline & 1010 \\ + & 1 \\ \hline & 1011 \end{array}$$

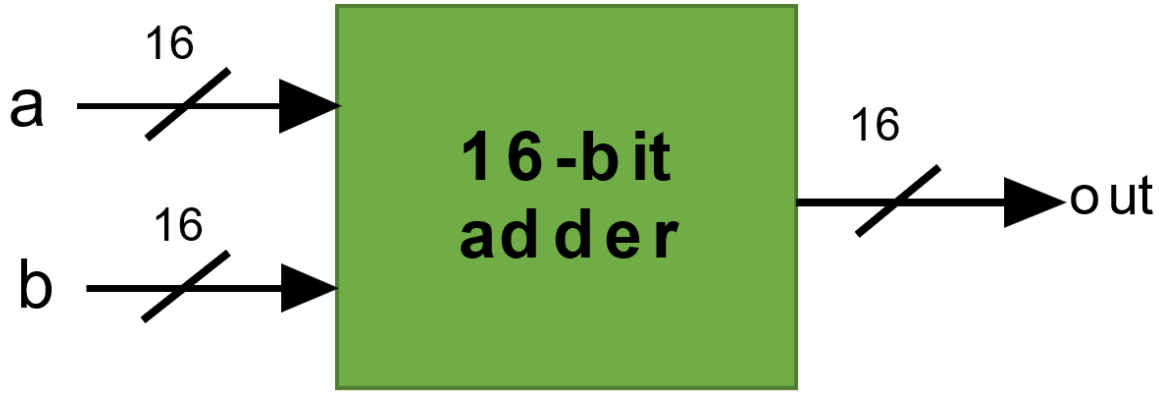
11

Example: 2 - 5 = 2 + (-5) =

$$\begin{array}{r} 0\,0\,1\,0 \\ +\,1\,0\,1\,1 \\ \hline 1\,1\,0\,1 \end{array} \quad = \quad -3$$

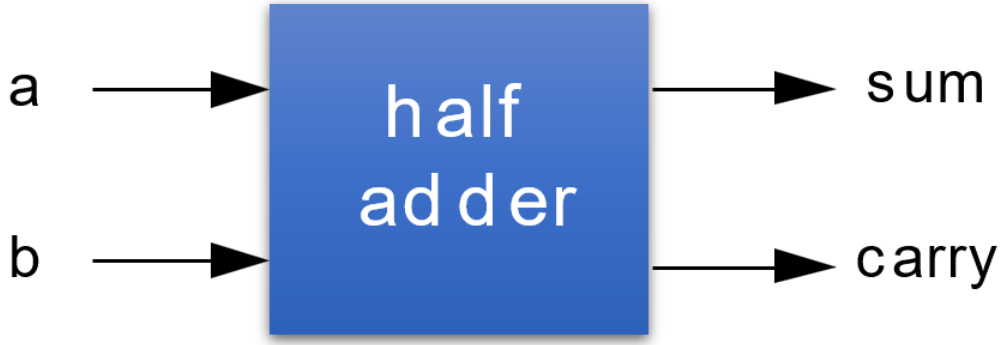


Building an Adder chip

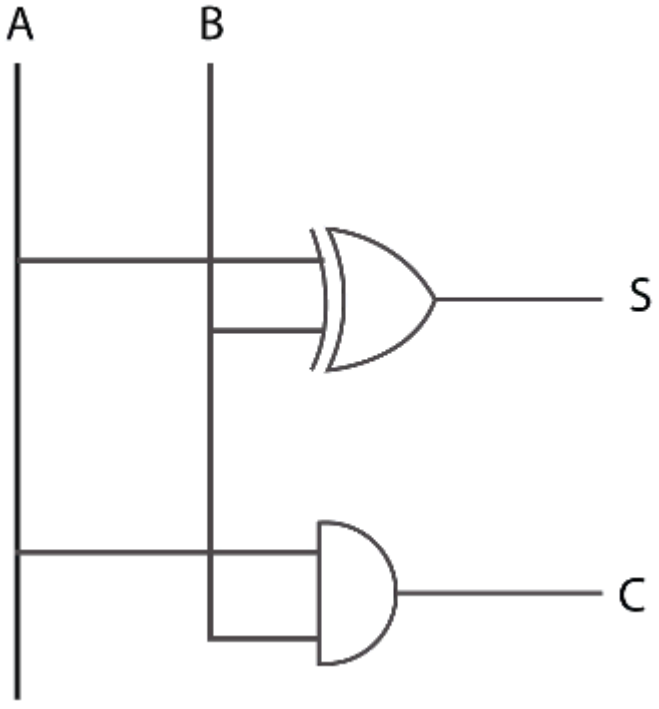


- Adder: a chip designed to add two integers
- Proposed implementation:
  - Half adder: designed to add 2 bits
  - Full adder: designed to add 3 bits
  - Adder: designed to add two *n*-bit numbers.

Half adder (designed to add 2 bits)



a	b	sum	carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



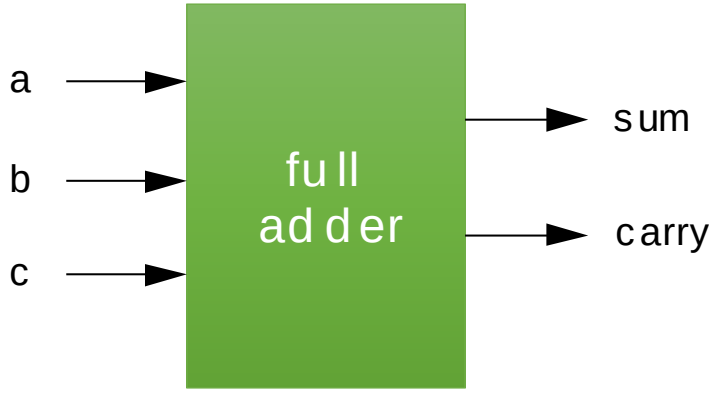
Logic Circuit Diagram

Implementation: based on two gates that you've seen before.



Full adder (designed to add 3 bits)

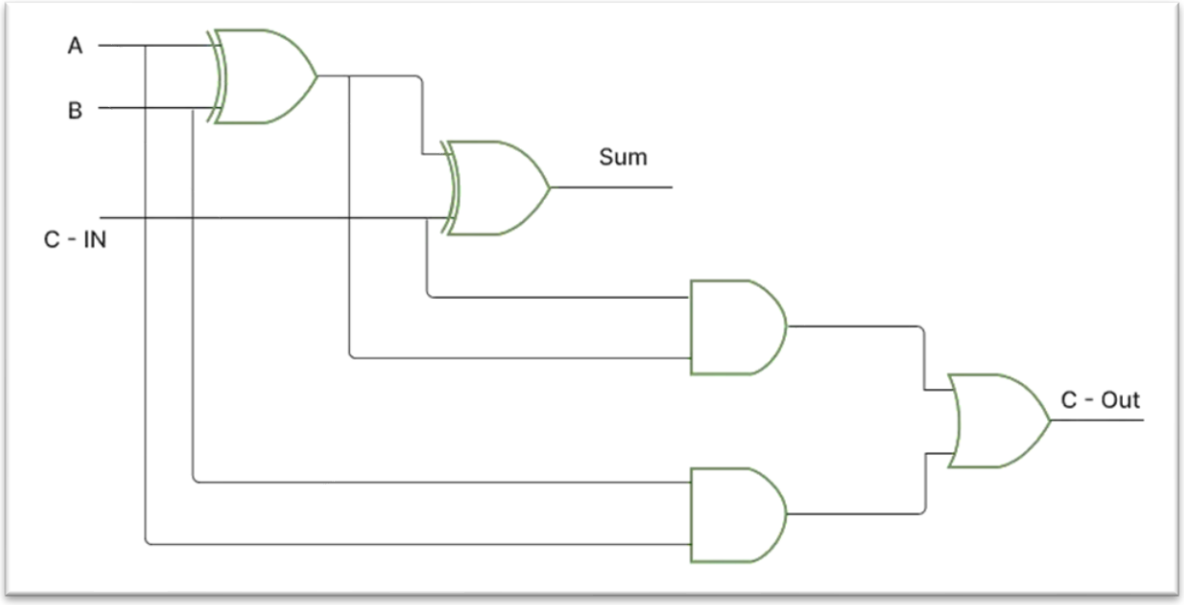
a	b	c	sum	carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



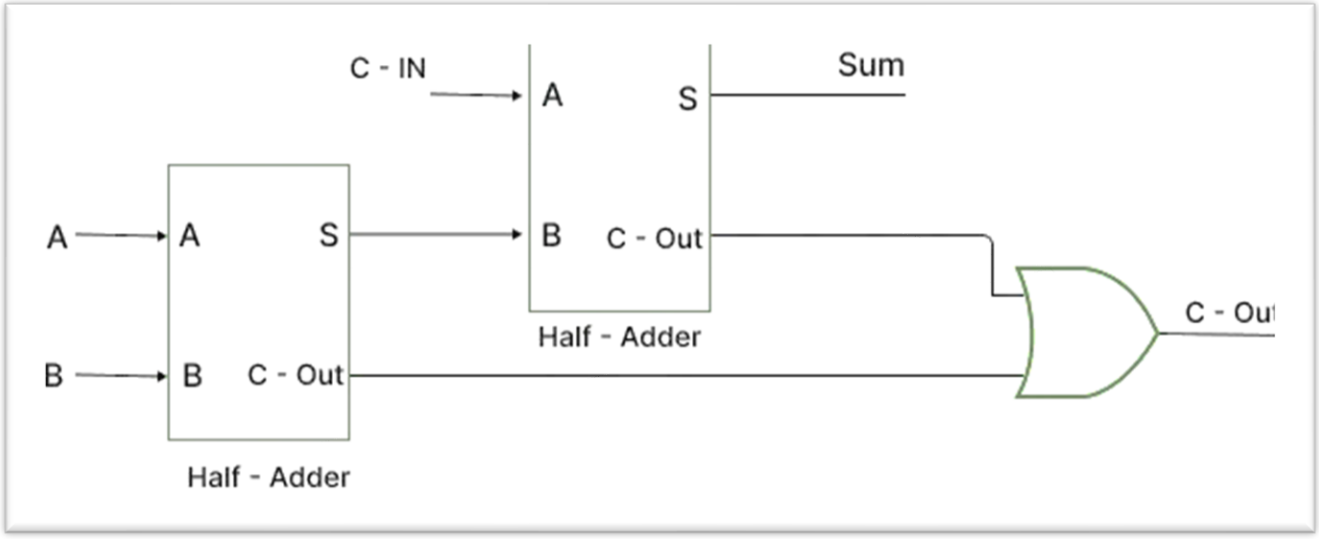
Implementation: can be based on half-adder gates.



Implementation of Full Adder



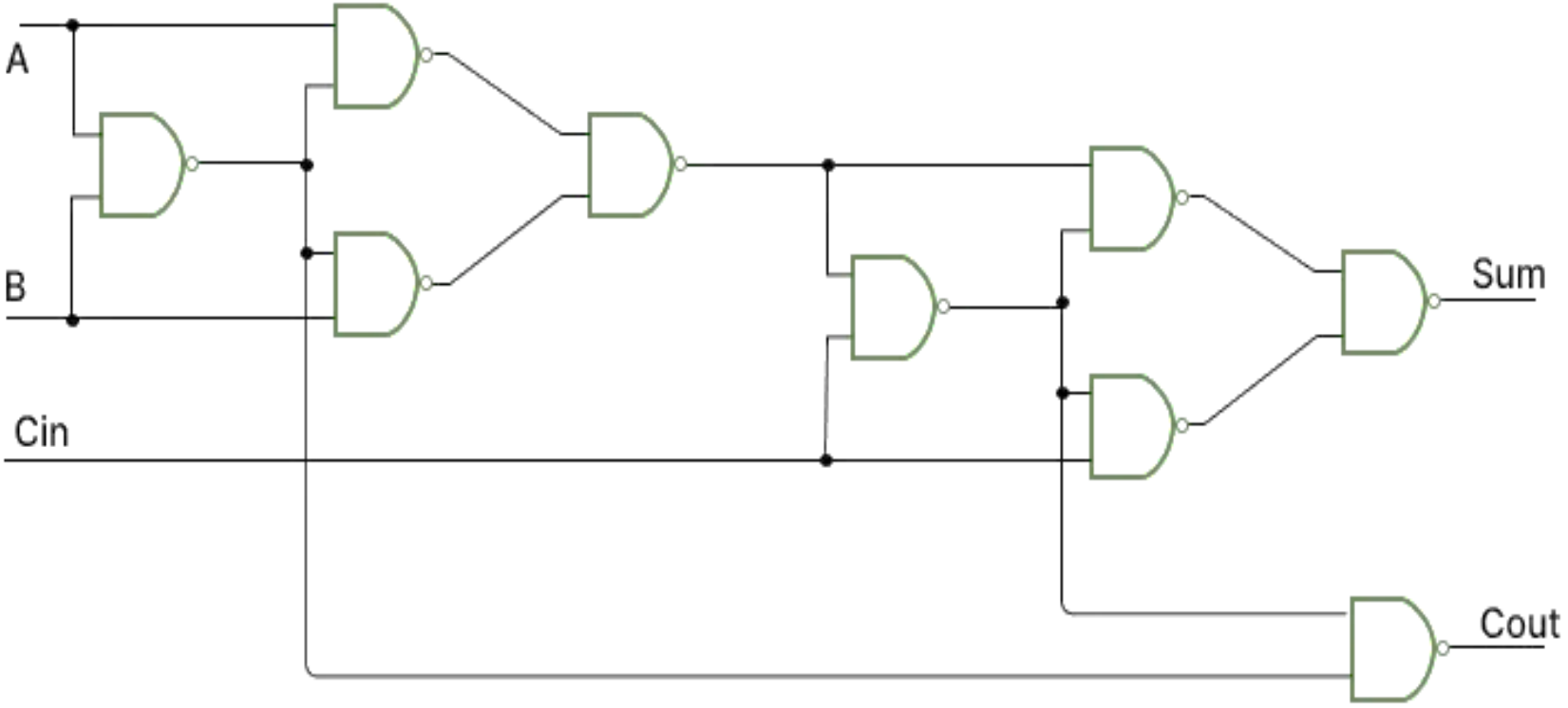
Implementation of Full Adder using Half Adders





Elements of Computing Systems, Nisan & Schocken, MIT Press, www.nand2tetris.org , Chapter 2: *Boolean Arithmetic*
https://www.geeksforgeeks.org/digital-logic/full-adder-in-digital-logic/

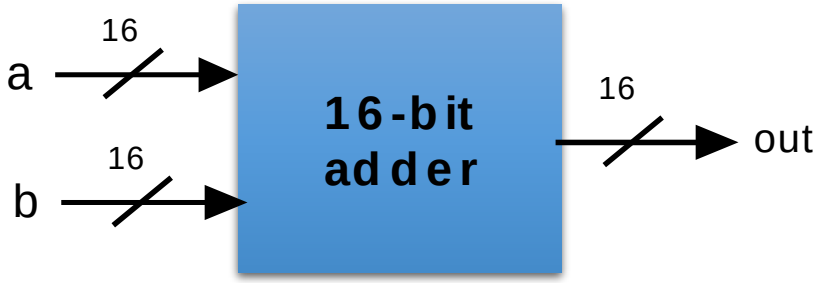
Implementation of Full Adder using NAND gates





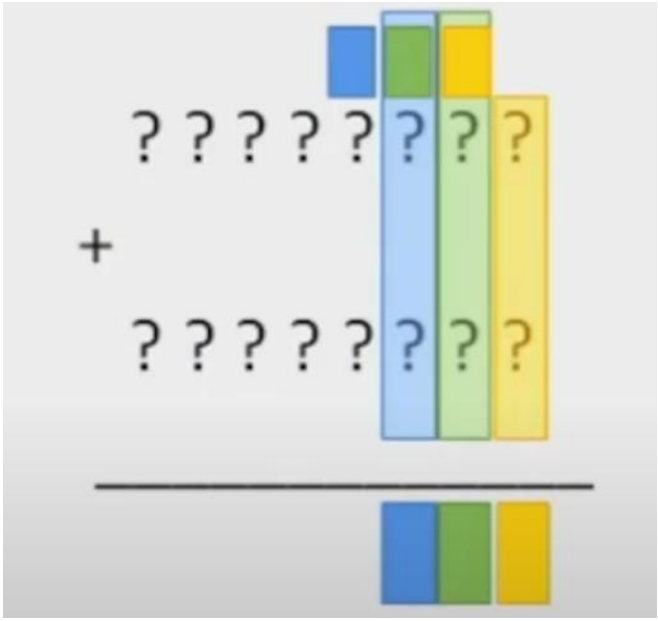
Elements of Computing Systems, Nisan & Schocken, MIT Press, www.nand2tetris.org , Chapter 2: *Boolean Arithmetic*
https://www.geeksforgeeks.org/digital-logic/full-adder-in-digital-logic/

n-bit Adder (designed to add two 16-bit numbers)



out=a+ b, as 16-bit integers(overflow ignored)

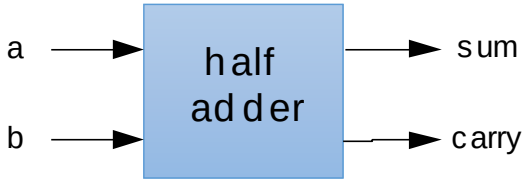


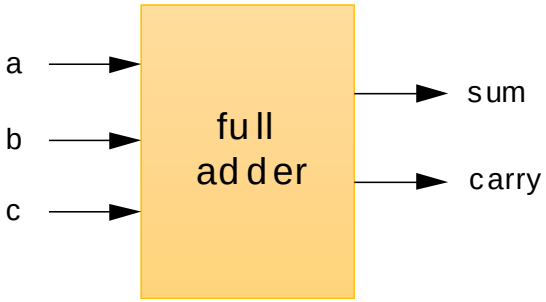


Implementation: array of full-adder gates.

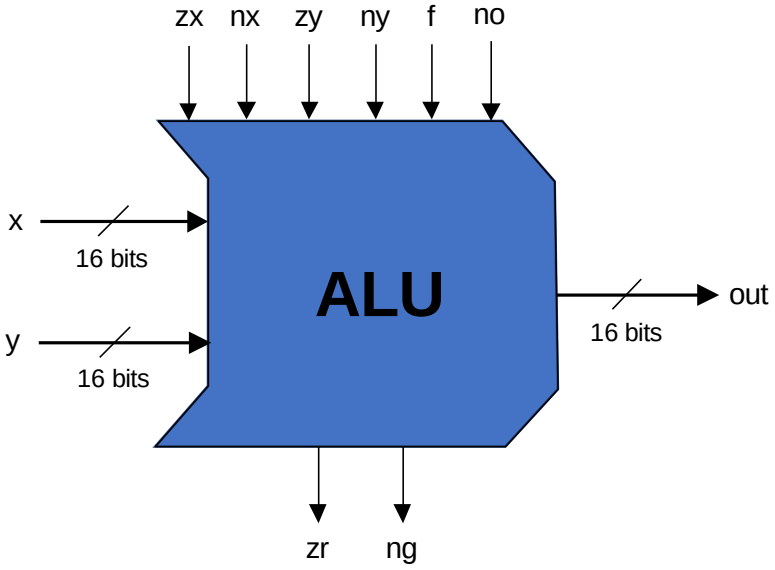


The ALU (of the Hack platform)









out(x, y,control bits) =

x+y, x-y, y–x,

0, 1, -1,

x, y, -x, -y, x!, y!,

x+1, y+1, x-1, y-1,

x&y, x|y

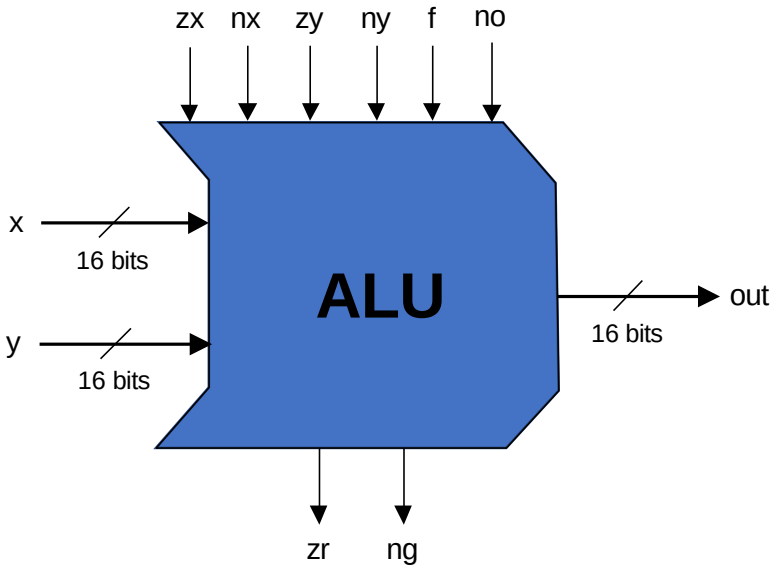


- Operates on two 16-bit, two's complement values
- Outputs a 16-bit, two's complement value
- Which function to compute is set by six 1-bit inputs
- Computes one out of a family of 18 functions

To cause the ALU to compute a function, set the control bits to the binary combination listed in the table.

control bits

zx	nx	zy	ny	f	no	out
1	0	1	0	1	0	0
1	1	1	1	1	1	1
1	1	1	0	1	0	-1
0	0	1	1	0	0	x
1	1	0	0	0	0	y
0	0	1	1	0	1	!x
1	1	0	0	0	1	!y
0	0	1	1	1	1	-x
1	1	0	0	1	1	-y
0	1	1	1	1	1	x+1
1	1	0	1	1	1	y+1
0	0	1	1	1	0	x-1
1	1	0	0	1	0	y-1
0	0	0	0	1	0	x+y
0	1	0	0	1	1	x-y
0	0	0	1	1	1	y-x
0	0	0	0	0	0	x&y
0	1	0	1	0	1	x|y



out(x, y,control bits) =

x+y, x-y, y–x,

0, 1, -1,

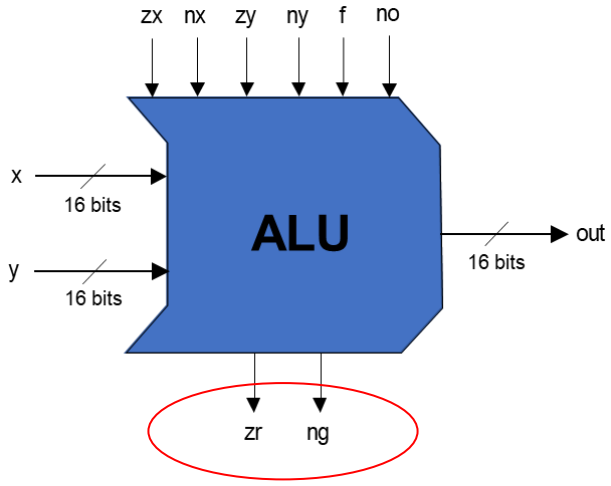
x, y, -x, -y, x!, y!,

x+1, y+1, x-1, y-1,

x&y, x|y

WOOSONG UNIVERSITY
Kazakhstan

Status Flags



- ZR and NG are status flags on the ALU's output. They aren't control bits you set; instead, the ALU sets them automatically after a calculation to give information about the result.
  - **zr (zero):** This flag is set to 1 if the ALU's output is zero. Otherwise, it's set to 0.
  - **ng (negative):** This flag is set to 1 if the ALU's output is negative. Otherwise, it's set to 0.
- These flags are used by the CPU to make decisions.



WOOSONG UNIVERSITY
Kazakhstan

ALU Inputs and Control Bits

- ALU has two main inputs, x and y, and a set of six control bits that modify those inputs and the final output.
- Control Bits for Input x
  - zx (zero x): If this bit is a 1, it forces the x input to become 0. If it's a 0, x remains unchanged.
  - nx (negate x): If this bit is a 1, it flips all the bits of the x input (performs a logical NOT). If it's a 0, x remains unchanged.
- Control Bits for Input y
  - zy (zero y): If this bit is a 1, it forces the y input to become 0.
  - ny (negate y): If this bit is a 1, it flips all the bits of the y input.
- Control Bit for the Function
  - f (function): This bit determines the operation the ALU performs on the modified x and y inputs.
    - If f = 1, the ALU performs addition (+).
    - If f = 0, the ALU performs a bitwise AND (&).
- Control Bit for the Output
  - no (negate output): If this bit is a 1, it flips all the bits of the final result.
  - If it's a 0, the output is unchanged.

control bits

zx	nx	zy	ny	f	no	out
1	0	1	0	1	0	0
1	1	1	1	1	1	1
1	1	1	0	1	0	-1
0	0	1	1	0	0	x
1	1	0	0	0	0	y
0	0	1	1	0	1	!x
1	1	0	0	0	1	!y
0	0	1	1	1	1	-x
1	1	0	0	1	1	-y
0	1	1	1	1	1	x+1
1	1	0	1	1	1	y+1
0	0	1	1	1	0	x-1
1	1	0	0	1	0	y-1
0	0	0	0	1	0	x+y
0	1	0	0	1	1	x-y
0	0	0	1	1	1	y-x
0	0	0	0	0	0	x&y
0	1	0	1	0	1	x|y

ALU logic (Hack platform)

These bits instruct how to pre-set the x input		These bits instruct how to pre-set the y input		This bit selects between + / And	This bit inst. how to post-set out	Resulting ALU output
zx	nx	zy	ny	f	no	out=
if zx then x=0	if nx then x=!x	if zy then y=0	if ny then y=!y	if f then out=x+y else out=x And y	if no then out=!out	f(x,y)=
1	0	1	0	1	0	0
1	1	1	1	1	1	1
1	1	1	0	1	0	-1
0	0	1	1	0	0	x
1	1	0	0	0	0	y
0	0	1	1	0	1	!x
1	1	0	0	0	1	!y
0	0	1	1	1	1	-x
1	1	0	0	1	1	-y
0	1	1	1	1	1	x+1
1	1	0	1	1	1	y+1
0	0	1	1	1	0	x-1
1	1	0	0	1	0	y-1
0	0	0	0	1	0	x+y
0	1	0	0	1	1	x-y
0	0	0	1	1	1	y-x
0	0	0	0	0	0	x&y
0	1	0	1	0	1	x|y

2^6 or **64 possible**

Implementation: build a logic gate architecture that "executes" the control bit "instructions": if zx==1 then set x to 0 (bit-wise), etc.



ALU logic (Hack platform)

These bits instruct how to pre-set the x input		These bits instruct how to pre-set the y input		This bit selects between + / And	This bit inst. how to post-set out	Resulting ALU output
zx	nx	zy	ny	f	no	out=
if zx then x=0	if nx then x=!x	if zy then y=0	if ny then y=!y	if f then out=x+y else out=x And y	if no then out=!out	f(x,y)=
1	0	1	0	1	0	0
1	1	1	1	1	1	1
1	1	1	0	1	0	-1
0	0	1	1	0	0	x
1	1	0	0	0	0	y
0	0	1	1	0	1	!x
1	1	0	0	0		
0	0	1	1	1		
1	1	0	0	1		
0	1	1	1	1		
1	1	0	1	1		
0	0	1	1	1		
1	1	0	0	1		
0	0	0	0	1		
0	1	0	0	1		
0	0	0	1	1		
0	0	0	0	0		
0	1	0	1	0		

2^6 or **64 possible**

Implementation: build a logic gate architecture that "executes" the control bit "instructions": if zx==1 then set x to 0 (bit-wise), etc.

Based on the control bits 001101, the ALU is set up to perform the operation **!x**

inputs x=01 and y=11.

Step 1: Modify the x-input
- **zx=0**: x is not zeroed.
- **nx=0**: x is not negated.
- The x input remains **01**.

Step 2: Modify the y-input
- **zy=1**: y is zeroed, becoming 00.
- **ny=1**: y is then negated, turning 00 into **11**.

Step 3: Perform the ALU Function
- **f=0**: The ALU performs a bitwise **AND** (&) on the modified inputs (01 and 11).
- 01 & 11 = 01

Step 4: Final Output
- **no=1**: The final output is negated.
- The result of the AND operation (01) is flipped, so it becomes **10**.
- The ALU's final output is **10**.

Example

Based on the control bits 001101, the ALU is set up to perform the operation **!x**

inputs x=01 and y=11.

If we input 2-bit numbers like 01 and 11 into a 16-bit ALU, the ALU will automatically pad them with leading zeros to make them 16 bits long.

- 01 becomes **0000 0000 0000 0001**
- 11 becomes **0000 0000 0000 0011**

Step 1: Modify the x-input
- **zx=0**: x is not zeroed.
- **nx=0**: x is not negated.
- The x input remains **01**.

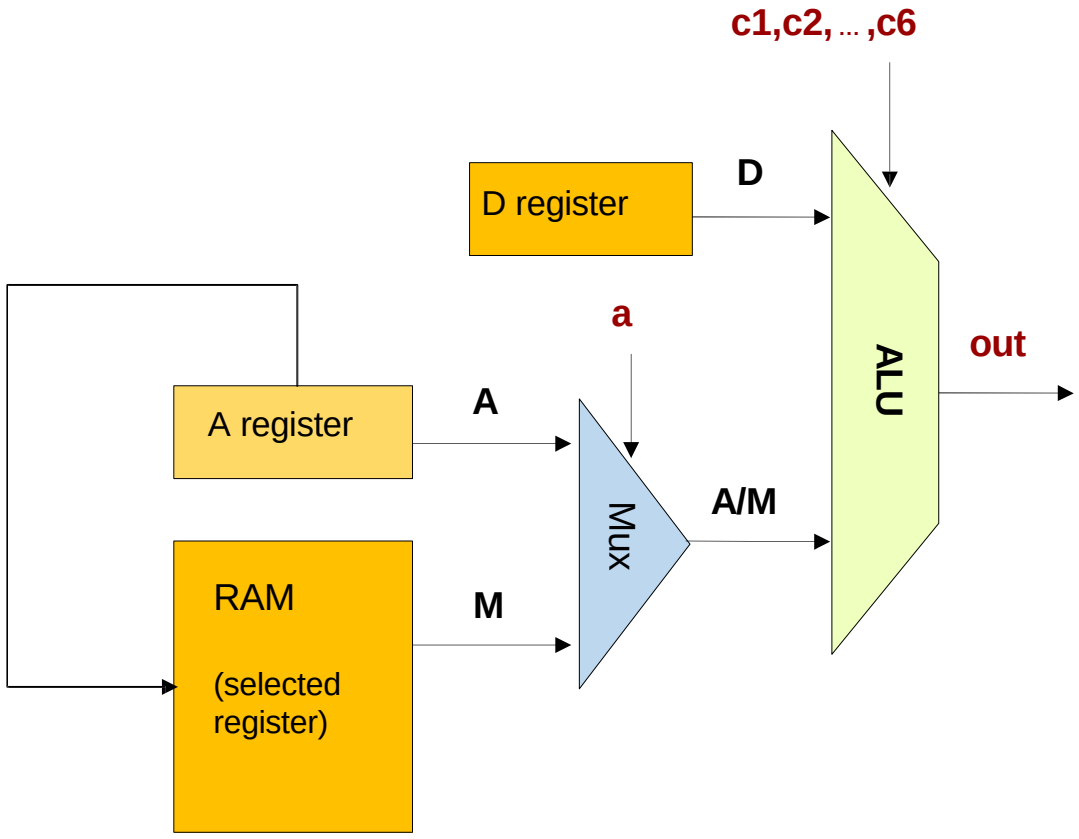
Step 2: Modify the y-input
- **zy=1**: y is zeroed, becoming 00.
- **ny=1**: y is then negated, turning 00 into **11**.

Step 3: Perform the ALU Function
- **f=0**: The ALU performs a bitwise **AND** (&) on the modified inputs (01 and 11).
- 01 & 11 = 01

Step 4: Final Output
- **no=1**: The final output is negated.
- The result of the AND operation (01) is flipped, so it becomes **10**.
- The ALU's final output is **10**.



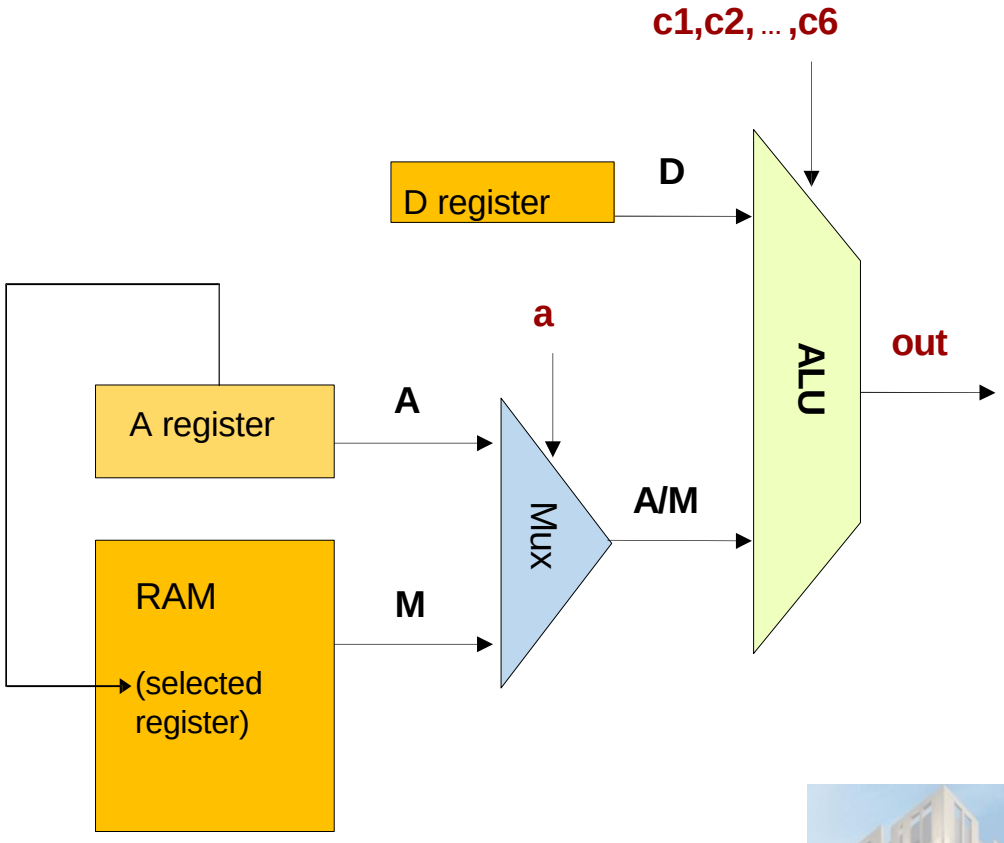
The ALU in the CPU context (a sneak preview of the Hack platform)



Data Storage (The Sources)

This is the **data path** shows how our computer's brain (the ALU) gets its numbers.
We have three main places where data comes from:

- **D Register:** (Point to D Register) Think of this as a temporary scratchpad. It holds the first number, D, for any calculation.
- **A Register:** (Point to A Register) This register does two jobs: it can hold a number, A, OR, more commonly, it holds the **address** that points to a specific location in memory.
- **RAM (Memory):** (Point to RAM) This is the main memory. The number stored inside RAM at the location pointed to by the A Register is called M."

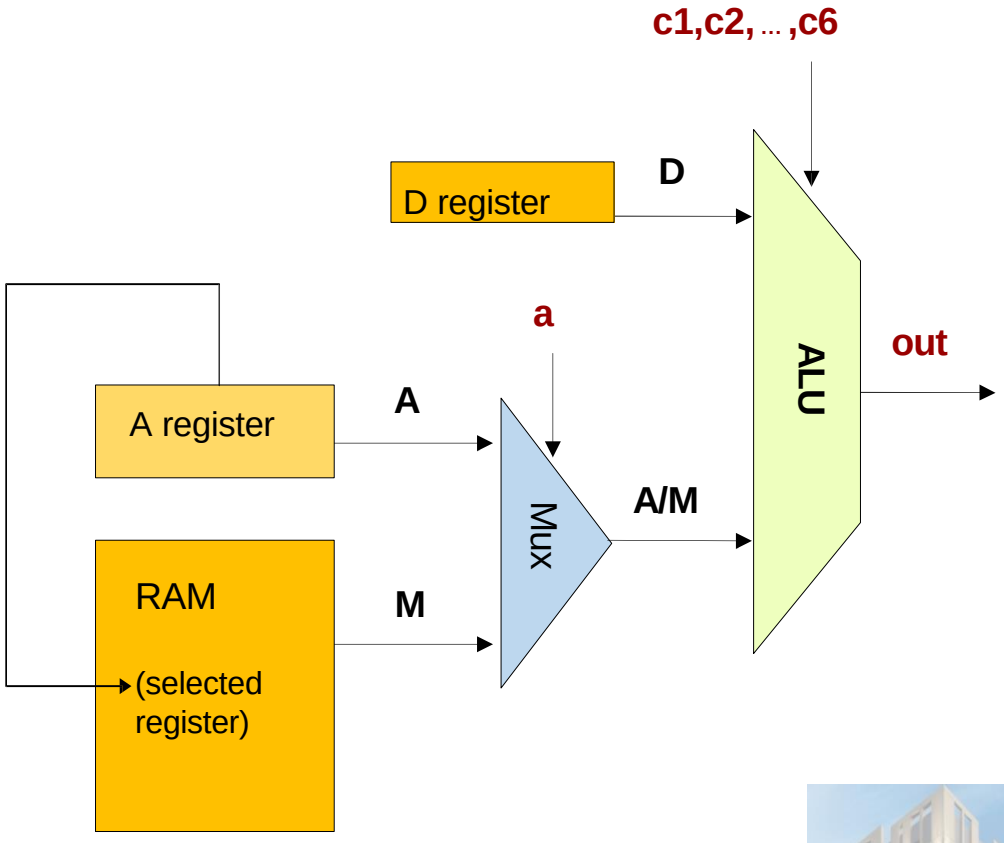




Data Selection (The Choice)

The ALU needs two inputs to calculate. It always gets **D** as one input, but the second input is a choice between ***A*** and **M**.

- **Mux (The Selector):** The blue triangle is a **Multiplexer**, or Mux. It's a switch that decides which number goes to the ALU.
- **Control Bit 'a':** (Point to 'a' input) We control the Mux with this single bit. If 'a' is 0, the Mux passes the number A. If 'a' is 1, it passes the number M."





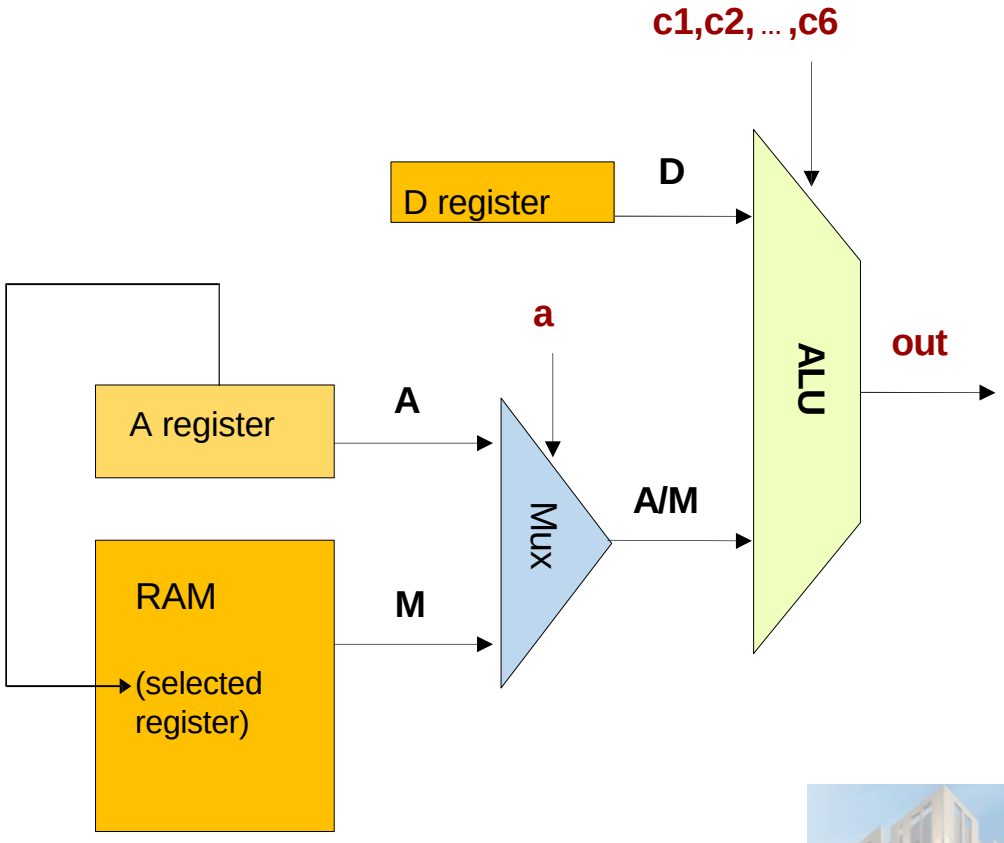
Data Selection (The Choice)

Arithmetic Logic Unit (ALU), is the calculator.

- **Inputs:** It takes the D value and the selected A/M value.

- **Control Bits (c_1–c_6):** (Point to the control bits) These six bits are the **command**. They tell the ALU exactly what to do. Should it add the numbers? Subtract them? Perform an AND operation? The programmer sets these bits to define the operation.

- **Output:** The out signal is the final answer, which can then be written back to the D register, the A register, or to the main RAM."





WOOSONG UNIVERSITY
Kazakhstan

Perspective

- Combinational logic
- Our adder design is very basic: no parallelism
- It pays to optimize adders
- Our ALU is also very basic: no multiplication, no division
- Where is the seat of more advanced math operations? a typical hardware/software tradeoff.

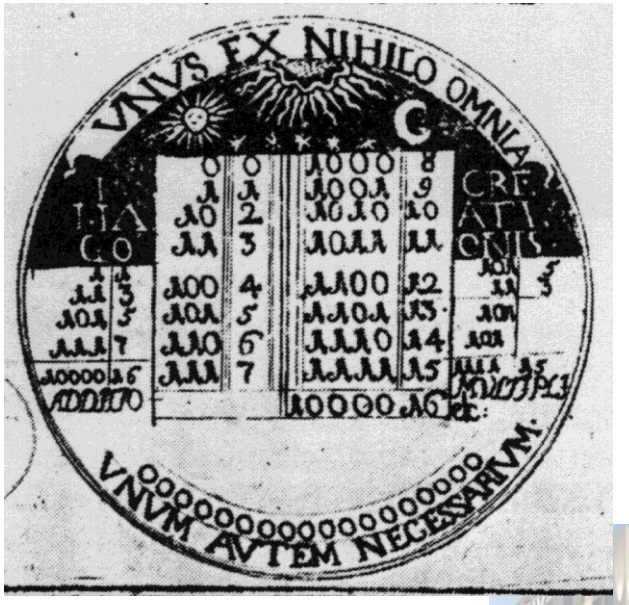


Historical end-note: Leibnitz (1646-1716)



- "The binary system may be used in place of the decimal system; express all numbers by unity and by nothing"
- 1679: built a mechanical calculator (+, -, *, /)





Leibniz's medallion for the Duke of Brunswick

- CHALLENGE: "All who are occupied with the reading or writing of scientific literature have assuredly very often felt the want of a common scientific language, and regretted the great loss of time and trouble caused by the multiplicity of languages employed in scientific literature:
- SOLUTION: "*Characteristica Universalis*": a universal, formal, and decidable language of reasoning
- The dream's end: Turing and Gödel in 1930's.